

GENZ Studios — AWS الكلاود شرح كامل للـ

Amazon Web Services مع خدمات GENZ Studios هذا الملف هو دليلك الشامل لفهم كيف يعمل تطبيق مكتوب بأسلوب بسيط جداً — كأنك بتشرح لطفل صغير لأول مرة يسمع عن الكلاود (AWS)

الفهرس

رقم	القسم
1	وليه بنستخدمه؟ AWS Amplify ما هو
2	خطوات إعداد الكلاود في التيرمينال خطوة بخطوة
3	جدول البيانات في السحابة — Schema الـ
4	(Cognito) نظام تسجيل الدخول — Authentication الـ
5	كيف نبعت ونستقبل البيانات — API (AppSync + GraphQL) الـ
6	تخزين الصور — Storage (S3) الـ
7	كيف يبدأ التطبيق — main.dart ملف
8	المحرك الأساسي — aws_storage.dart ملف
9	(Mutations) كيف تبعت البيانات
10	(Queries) كيف تستقبل البيانات
11	الـ Real-Time (Subscriptions) الـ
12	نظام الإشعارات بالتفصيل
13	نظام الشات بالتفصيل
14	مش شغالة Subscriptions البديل لما الـ Polling الـ

1. 🌩️ وليه بنستخدمه؟ AWS Amplify ما هو

...تخيل معايا القصة دي

:عندك زبائن بيجوا كل يوم، الزبون ده محتاج (GENZ Studios تطبيق) تخيل إنك عندك محل كبير

- (Authentication) **يسجل اسمه** عند الدخول
- (Database) **يحجز استوديو** ويحتفظ بالحجز
- (Storage) **يرفع صورته** وصور الاستوديو
- (Chat) **يتكلم مع الموظف** في real-time
- (Notifications) **يستلم إشعارات** لما حجزه يتقبل أو يترفض

لو هتعمل كل ده بنفسك — هتحتاج تشتري سيرفرات، تركيبها، تحميها، وتدفع فلوس تانية على الكهرباء والصيانة... ده صعب جداً وغالي!

بتأجرك "كمبيوترات ضخمة في السحابة" وأنت بس بتدفع على Amazon: هو ببساطة **AWS (Amazon Web Services)** اللي بتستخدمه. مفيش سيرفرات تشتريها، مفيش صيانة، مفيش ضغط.

بالتحديد؟ Amplify وما هو

في تطبيقك. بدل ما تكتب مئات AWS ببسّهل عليك استخدام خدمات Amazon هو "مساعد ذكي" من **AWS Amplify**. بتكتب أوامر بسيطة وهو بيعمل كل حاجة تلقائياً — AWS الأسطر من الكود لتتصل بـ

بدون Amplify:	Amplify مع:
أوامر بسيطة	→ كود معقد جداً
إعداد تلقائي	→ إعداد يدوي
دقائق معدودة	→ وقت طويل

GENZ Studios لـ Amplify ليه اخترنا

السبب	التفسير
سهولة الاستخدام	أوامر بسيطة في التيرمينال تعمل كل حاجة
Flutter Support	جاهزة packages بشكل ممتاز مع Flutter يدعم
Real-time	refresh يعني بيانات مباشرة بدون subscriptions يدعم
جاهز Authentication	نظام تسجيل دخول كامل بدون ما تكتب حاجة
مجاني في البداية	يكفي للتطوير والاختبار Free tier
Scalable	بيكبر معاك تلقائياً AWS — لو طلب عليك كتير

GENZ الخدمات الرئيسية اللي بنستخدمها في

GENZ Studios App

🔒 Amazon Cognito	→ تسجيل الدخول والخروج
🌐 AWS AppSync	→ الـ API (GraphQL)
📦 Amazon DynamoDB	→ قاعدة البيانات (خلف الكواليس)
📁 Amazon S3	→ تخزين الصور
⚡ GraphQL Subscriptions	→ البيانات الحية (Real-time)

مساعد ببسّهل = Amplify. بتأجرك كمبيوترات وخدمات في السحابة Amazon شركة = AWS. **ملخص القسم الأول** AppSync (API) + (دخول) Cognito بيستخدم GENZ Studios. في تطبيقك AWS استخدام (صور) S3.

2. خطوات إعداد الكلاود في التيرمينال خطوة بخطوة 🖥️

ما هو التيرمينال؟

هو النافذة السوداء التي بتكتب فيها أوامر نصية. بدل ما تضغط (PowerShell أو Command Prompt أو التيرمينال بالماوس — بتكتب أوامر وهي بتنفذ

المتطلبات قبل ما تبدأ

قبل أي حاجة، لازم يكون عندك:

1. **Node.js** (من nodejs.org) مثبت على الجهاز
2. **AWS** حساب (من aws.amazon.com — مجاني)
3. **Flutter** مثبت
4. **VS Code** أو أي محرر كود

الأوامر خطوة بخطوة

Amplify CLI الخطوة 1: تثبيت

```
npm install -g @aws-amplify/cli
```

شرح سطر سطر:

- **npm** = مدير الحزم لـ Node.js (زي متجر تطبيقات للكود)
- **install** = حمّل وثبّت
- **-g** = globally يعني على كل الجهاز مش بس في الفولدر الحالي
- **@aws-amplify/cli** = اسم الحزمة اللي عايز تثبتها

في أي تيرمينال **amplify** على جهازك، وبعدها تقدر تكتب كلمة **amplify** ماذا يحدث؟ بتحمّل برنامج اسمه

للتأكد إنه اتثبت صح:

```
amplify --version
# X.X. هيطلع مثلاً: 12
```

AWS Credentials الخطوة 2: إعداد

```
amplify configure
```

شرح سطر سطر:

- **amplify** = الأداة اللي حملتها في الخطوة السابقة
- **configure** = اعدّ نفسك "يعني قوله بياناتك"

ماذا يحدث؟ هيفتح المتصفح تلقائياً وهيطلب منك

1. الخاص بك AWS تسجيل الدخول لحساب
2. IAM جديد (IAM = Identity and Access Management) إنشاء مستخدم
3. إعطاء الأوامر صلاحيات للتعامل مع حسابك

مثال على ما ستراه في التيرمينال

```
Specify the AWS Region: us-east-1
Specify the username of the new IAM user: genz-amplify-user
Complete the user creation using the AWS console...
Enter the access key of the newly created user:
accessKeyId: [تكتب هنا]
secretAccessKey: [تكتب هنا]
```

لا تشاركهمش مع أي حد ولا AWS. هما كلمة سر حسابك على Secret Key و Access Key ملاحظة مهمة: ال
!تحطهمش في الكود

الخطوة 3: تهيئة المشروع

```
amplify init
```

شرح:

- `init` = initialize "أبدأ من الأول"

ماذا يحدث؟ هيسألك أسئلة عن مشروعك

```
Enter a name for the project: genz
Enter a name for the environment: dev
Choose your default editor: Visual Studio Code
Choose the type of app that you're building: flutter
Where do you want to store your configuration file? ./lib/
```

بعد ما تجاوب هيتعمل

```
amplify/
├── backend/           ← إعدادات ال backend
├── .config/           ← إعدادات المشروع
└── team-provider-info.json
```

وده أهم ملف — بيحتوي على كل عناوين ال `/lib/` في `amplifyconfiguration.dart` هيتعمل ملف Flutter وفي API وال credentials.

Authentication الخطوة 4: إضافة

```
amplify add auth
```

شرح:

- **add** = أضف خدمة جديدة
- **auth** = Authentication (نظام التحقق من الهوية)

ماذا يحدث؟ هيسألك:

```
Do you want to use the default authentication and security configuration?  
▸ Default configuration  
  Default configuration with Social Provider (Federation)  
  Manual configuration  
  I want to learn more.
```

اخترنا الإعدادات دي GENZ في

```
userPoolName: genz47b9027f_userpool_47b9027f  
autoVerifiedAttributes: email # بيثبت كود تأكيد على الإيميل  
mfaConfiguration: OFF # بدون Two-Factor Authentication  
passwordPolicyMinLength: 8 # الباسورد 8 حروف على الأقل  
usernameAttributes: email # الدخول بالإيميل (مش يوزنيم)  
usernameCaseSensitive: false # test@gmail.com = TEST@gmail.com  
authSelections: identityPoolAndUserPool # نوع خاص من الـ Auth
```

API الخطوة 5: إضافة

```
amplify add api
```

شرح:

- **api** = Application Programming Interface يعني "الطريق لتبادل البيانات"

ماذا يحدث؟ هيسألك:

```
Select from one of the below mentioned services:  
▸ GraphQL  
  REST
```

Here is the GraphQL API that we will create. Select a setting to edit or continue:

- Name: genz
- Authorization modes: Amazon Cognito User Pool (default)
- Conflict detection (required for DataStore): Enabled
- Continue

اخترنا GENZ في

- **GraphQL** (بدل REST) — لأنه أسرع وأكفأ
- **Conflict Detection: Enabled** — نحل التعارض، نحل نفس الوقت، نحل التعارض
- **Authorization: Cognito** — بس الناس المسجلين يقدروا يوصلوا البيانات

Storage الخطوة 6: إضافة

```
amplify add storage
```

شرح:

- **storage** = تخزين الملفات والصور

ماذا يحدث؟ هيسألك:

Select from one of the below mentioned services:

- Content (Images, audio, video, etc.)
- NoSQL Database

Who should have access:

- Auth users only
- Auth and Guest users

What kind of access do you want for Authenticated users?

- create/update, read, delete

بس المستخدمين المسجلين يقدروا يرفعوا صور GENZ في.

الخطوة 7: رفع التغييرات للسحابة

```
amplify push
```

شرح:

- **push** = AWS ادفع" يعني ارفع كل الإعدادات اللي عملتها لـ"

ماذا يحدث؟ ده أهم أمر! هيعمل التالي

- ✓ إنشاء User Pool في Cognito
- ✓ إنشاء GraphQL API في AppSync
- ✓ إنشاء جداول في DynamoDB
- ✓ إنشاء Bucket في S3
- ✓ ربط كل الخدمات ببعض

هيسألك:

```
Do you want to generate code for your newly created GraphQL API? Yes
Choose the code generation language target: dart
Enter the file name pattern of graphql queries...: lib/graphql/**/*.graphql
Do you want to generate/update all possible GraphQL operations?: Yes
```

يبيني كل حاجة في السحابة AWS ده بياخد وقت (5-15 دقيقة) عشان

Dart Models الخطوة 8: توليد الـ

```
amplify codegen models
```

شرح:

- **codegen** = code generation تلقائياً
- **models** = النماذج (الـ classes في Dart)

تلقائياً Dart ويولد ملفات (اللي هنشره بعدين) Schema **ماذا يحدث؟** هيقرأ الـ

```
lib/models/
├── Studio.dart
├── BookingRequest.dart
├── ChatMessage.dart
├── AppNotification.dart
├── UserProfile.dart
└── ModelProvider.dart
```

Schema بيكتبها تلقائياً من الـ Amplify، بإيدك classes ده بيوفر عليك ساعات من الكتابة! بدل ما تكتب الـ

AppSync Console الخطوة 9: فتح الـ

```
amplify console api
```

شرح:

- `console` = افتح لوحة التحكم في المتصفح
- `api` = تحديد API للـ

حيث تقدر AWS في AppSync **ماذا يحدث؟** هيفتح المتصفح مباشرة على صفحة

- تشوف جداول البيانات
- يدويًا GraphQL Queries تجرب الـ
- Subscriptions تشوف الـ

الخطوة 10: عرض حالة المشروع

```
amplify status
```

شرح:

- `status` = "ما هو الوضع الحالي؟"

مثال على الناتج:

Current Environment: dev

Category	Resource name	Operation	Provider plugin
Auth	genz47b9027f	No Change	awscloudformation
Api	genz	No Change	awscloudformation
Storage	s3bucket	No Change	awscloudformation

هو الأمر الأهم — هو اللي `amplify push`. **ملخص القسم الثاني:** الأوامر دي بتبني البنية التحتية الكاملة لتطبيقك في AWS. الترتيب مهم: `configure` → `init` → `add auth` → `add api` → `add storage` → `push` → `codegen models`. بيرفع كل حاجة للسحابة فعلاً.

3. Schema — جدول البيانات في السحابة

Schema ما هو الـ

هو "قالب الدفتر" — بيحدد Schema تخيل إنك عندك دفتر لتسجيل حجوزات الاستوديو. الـ

- إيه الأعمدة الموجودة (الحقول)
- نوع كل عمود (نص، رقم، صواب/خطأ)
- هل الحقل إجباري أم اختياري

محفوظ في Schema الـ GENZ، في


```
amplify/backend/api/genz/schema.graphql
```

شرح GraphQL Schema

يُعمل GraphQL — عنده أنواع بيانات (نص، رقم، تاريخ) Excel هي لغة تستخدمها لوصف شكل البيانات. زي ما **GraphQL** يعمل API نفس الحاجة بس للـ

(النماذج الـ 5) GENZ جداول البيانات في

📋 (الاستوديو) Studio: الجدول الأول

```
type Studio @model @auth(rules: [{ allow: private }]) {
  id: ID!
  name: String!
  type: String!
  pricePerHour: Int!
  description: String
  image: String
  available: Boolean
}
```

شرح كل سطر:

السطر	المعنى
<code>type Studio</code>	"Studio" اسم الجدول هو
<code>@model</code>	"لده DynamoDB اعمل جدول في" Amplify كلمة سحرية تقول لـ
<code>@auth(rules: [{ allow: private }])</code>	يقدرُوا يشوفُوا البيانات (private) بس الناس المسجلين
<code>id: ID!</code>	"فريد — الـ ! معناها" إجباري ID كل استوديو له
<code>name: String!</code>	اسم الاستوديو — نص إجباري
<code>type: String!</code>	نوع الاستوديو (تسجيل صوتي، تصوير...) — إجباري
<code>pricePerHour: Int!</code>	السعر بالساعة — رقم صحيح إجباري
<code>description: String</code>	وصف الاستوديو — اختياري (بدون !)
<code>image: String</code>	اختياري — S3 رابط الصورة في
<code>available: Boolean</code>	false أو true هل متاح؟

مثال على البيانات:

```
{
  "id": "studio-001",
```

```

"name": "استوديو النجوم",
"type": "تسجيل صوتي",
"pricePerHour": 150,
"description": "استوديو مجهز بالكامل",
"image": "s3://genz-bucket/studios/studio-001.jpg",
"available": true
}

```

الجدول الثاني: BookingRequest (طلب الحجز)

```

type BookingRequest @model @auth(rules: [{ allow: private }]) {
  id: ID!
  clientEmail: String! @index(name: "byClientEmail", queryField:
"bookingRequestsByClientEmail")
  clientName: String
  clientPhone: String
  studio: String!
  date: String!
  hours: String!
  price: String!
  equipment: String
  status: String
  fullStartDateTime: String! @index(name: "byStartDate", queryField:
"bookingRequestsByStart")
  fullEndDateTime: String!
}

```

شرح كل سطر:

الحقل	النوع	المعنى
id	ID	رقم فريد لكل حجز
clientEmail	String!	إيميل العميل — إجباري
clientName	String	اسم العميل — اختياري
clientPhone	String	تليفون العميل — اختياري
studio	String!	اسم الاستوديو المحجوز — إجباري
date	String!	تاريخ الحجز — إجباري
hours	String!	عدد الساعات المحجوزة — إجباري
price	String!	السعر الإجمالي — إجباري
equipment	String	معدات إضافية مطلوبة — اختياري
status	String	حالة الحجز: Pending/Approved/Rejected

الحقل	النوع	المعنى
<code>fullStartDateTime</code>	String!	وقت البداية الكامل — إجباري
<code>fullEndTimeTime</code>	String!	وقت النهاية الكامل — إجباري

ما هو `@index`؟

```
clientEmail: String! @index(name: "byClientEmail", queryField:
"bookingRequestsByClientEmail")
```

يُعمل "فهرس" على الحقل ده. تخيل كتاب فيه فهرس في الآخر — بدل ما تقلب كل الصفحات لتلاقي موضوع، `@index` ال بتبص في الفهرس مباشرة.

- `name: "byClientEmail"` = اسم الفهرس
- `queryField: "bookingRequestsByClientEmail"` = ال `query` كود اسم ال

"مباشرة" هاتلي حجوزات العميل ده بس **AWS** يعني بدل ما تجيب كل الحجوزات وتفلترها في الكود — بتطلب من

رسالة الشات: `ChatMessage`: الجدول الثالث

```
type ChatMessage @model @auth(rules: [{ allow: private }]) {
  id: ID!
  senderName: String
  senderEmail: String
  clientEmail: String! @index(name: "byClientEmail", queryField:
"chatMessagesByClientEmail")
  text: String
  time: String
  messageType: String
  parentId: String @index(name: "byParent", queryField: "chatMessagesByParent")
}
```

شرح الحقول:

الحقل	المعنى
<code>senderName</code>	اسم اللي بعث الرسالة
<code>senderEmail</code>	إيميل المُرسِل
<code>clientEmail</code>	إيميل العميل المرتبط بالمحادثة (مش بالضرورة المُرسِل)
<code>text</code>	نص الرسالة
<code>time</code>	وقت الإرسال
<code>messageType</code>	نوع الرسالة: "text"، "image"، "system"

الحقل	المعنى
parentId	الرسالة الأصلية ID — لو الرسالة رد على رسالة ثانية

للفهرس؟ senderEmail مش clientEmail لماذا

عشان نجيب كل رسائل محادثة عميل clientEmail لأن المحادثة دائماً بين عميل واحد وكل الموظفين. فينفهرس على معين بسرعة.

🔔 الإشعار: AppNotification: الجدول الرابع

```
type AppNotification @model @auth(rules: [{ allow: private }]) {
  id: ID!
  clientEmail: String! @index(name: "byClientEmail", queryField: "appNotificationsByClientEmail")
  title: String
  body: String
  type: String
  time: String
  read: Boolean
}
```

شرح الحقول:

الحقل	المعنى	مثال
clientEmail	مين المفروض يستقبل الإشعار	ahmed@gmail.com
title	عنوان الإشعار	"!تم قبول حجزك"
body	نص الإشعار	"...تم تأكيد حجز استوديو النجوم"
type	نوع الإشعار	"Approved", "Rejected", "Pending", "chat_opened"
time	وقت الإشعار	"2024-01-15T10:30:00Z"
read	هل قراه العميل؟	true / false

👤 الملف المستخدم: UserProfile: الجدول الخامس

```
type UserProfile @model @auth(rules: [{ allow: private }]) {
  id: ID!
  email: String! @index(name: "byEmail", queryField: "userProfilesByEmail")
  name: String
  image: String
  type: String
  chatEnabled: Boolean
}
```

```
lastUpdated: AWSDateTime
}
```

شرح الحقول:

الحقل	المعنى	مثال
email	إيميل المستخدم	ahmed@gmail.com
name	الاسم الكامل	"Ahmed Mohamed"
image	صورة الملف الشخصي	S3 رابط
type	نوع المستخدم	"client" أو "employee"
chatEnabled	هل الشات مفتوح لهذا العميل؟	true / false
lastUpdated	آخر تعديل	AWSDateTime نوع خاص بـ AWS

مثلاً: ISO 8601 يحفظ التاريخ والوقت بتنسيق AWS AppSync ده نوع بيانات خاص بـ **AWSDateTime** ملاحظة على `15-01-2024T10:30:00.000Z`).

Amplify الحقول التلقائية اللي بيضيفها

بيضيف حقول إضافية تلقائياً لكل جدول Amplify، `@model` لما بتعمل:

```
_version: Int!          # رقم الإصدار (لـ conflict detection)
_deleted: Boolean       # تحذف record؟ هل الـ (soft delete)
_lastChangedAt: AWSTimestamp! # آخر تعديل
_createdAt: AWSDateTime!   # تاريخ الإنشاء
_updatedAt: AWSDateTime!   # تاريخ آخر تعديل
```

Mutations. هنشرحه بالتفصيل في قسم الـ **_version**: أهمهم

بيعمل `@model` هو "خريطة البيانات" — بيحدد شكل كل جدول وحقوله. الـ Schema **ملخص القسم الثالث**: الـ Studio، بيعمل فهرس للبحث السريع على حقل معين. عندنا 5 جداول `@index` تلقائياً. الـ DynamoDB جدول في `BookingRequest`, `ChatMessage`, `AppNotification`, `UserProfile`.

4. 🔑 Authentication — (Cognito) نظام تسجيل الدخول

Cognito ما هو؟

هو نظام التحقق من الهوية **Amazon Cognito**. تخيل الكونسيرج في الفندق — اللي بيسألك "مين حضرتك؟" قبل ما تدخل في AWS.

بيعمل:

- تسجيل حسابات جديدة (Sign Up)
- تسجيل الدخول (Sign In)

- تسجيل الخروج (Sign Out)
- تأكيد الإيميل (Email Verification)
- استعادة الباسورد (Forgot Password)

GENZ في Cognito إعدادات

```
userPoolName: genz47b9027f_userpool_47b9027f
autoVerifiedAttributes: email
mfaConfiguration: OFF
passwordPolicyMinLength: 8
usernameAttributes: email
usernameCaseSensitive: false
authSelections: identityPoolAndUserPool
```

شرح كل إعداد

userPoolName: genz47b9027f_userpool_47b9027f

؟ تخيله كدفتر أسماء — فيه كل حسابات مستخدمى التطبيق **User Pool** ما هو

- **genz** = اسم المشروع
- **47b9027f** = عشان يضمن الاسم فريد Amplify رقم عشوائي أضافه
- **_userpool_** = ده User Pool
- خاص بيه User Pool له Amplify كل مشروع

autoVerifiedAttributes: email

بيبعثله إيميل فيه كود تأكيد. لازم يكتب الكود عشان يكمل التسجيل Amplify، **معناه:** لما حد يسجل حساب جديد

```
المستخدم يكتب إيميله وباسورده
↓
يبعث إيميل بكود (مثلاً: 847293) Cognito
↓
المستخدم يكتب الكود فى التطبيق
↓
الحساب يتفعل
```

ليه ده مهم؟ عشان نتأكد إن الإيميل حقيقي وبتاع المستخدم فعلاً

mfaConfiguration: OFF

MFA = Multi-Factor Authentication (التحقق بخطوتين — زي ما بيعمل البنك)

يعني مطلوبش. المستخدم بيدخل إيميل + باسورد بس GENZ: OFF في

كمان (أمان أكثر بس أصعب على المستخدمين) SMS كان هيبعت كود: ON لو كانت

passwordPolicyMinLength: 8

ببرفض الباسورد لو أقصر من كده Cognito. الباسورد لازم يكون 8 حروف على الأقل.

يمكن تضيف شروط تانية زي:

```
requireUppercase: true    # لازم فيه حرف كبير
requireNumbers: true     # لازم فيه رقم
requireSymbols: true     # لازم فيه رمز (!@#$)
```

usernameAttributes: email

منفصل username يعني المستخدم بيسجل الدخول بـ الإيميل مش بـ

```
بدل ما:  username: ahmed123
         password: xxxxxxxx

بيكون:  email: ahmed@gmail.com
         password: xxxxxxxx
```

أسهل للمستخدم عشان الإيميل بيتذكره دائماً

usernameCaseSensitive: false

يعني مفيش فرق بين الحروف الكبيرة والصغيرة في الإيميل

```
Ahmed@Gmail.com = ahmed@gmail.com = AHMED@GMAIL.COM
```

authSelections: identityPoolAndUserPool

ده إعداد متقدم شوية

المصطلح	المعنى
User Pool	"ببتحكم في" مين مسموح يدخل التطبيق
Identity Pool	(S3 زي) AWS services بيدي كل مستخدم صلاحيات مؤقتة للـ

بالتالي:

- User Pool = "هل هذا المستخدم موجود في نظامنا؟"
- Identity Pool = "ما هي صلاحياته في AWS؟"

كيف يبدو تسجيل الدخول في الكود

```
// تسجيل الدخول
Future<void> signIn(String email, String password) async {
  try {
    final result = await Amplify.Auth.signIn(
      username: email,
      password: password,
    );
    if (result.isSignedIn) {
      print('تم تسجيل الدخول بنجاح');
    }
  } on AuthException catch (e) {
    print('خطأ: ${e.message}');
  }
}
```

```
// التسجيل الجديد
Future<void> signUp(String email, String password) async {
  await Amplify.Auth.signUp(
    username: email,
    password: password,
    options: SignUpOptions(
      userAttributes: {
        AuthUserAttributeKey.email: email,
      },
    ),
  );
}
```

```
// تأكيد الإيميل
Future<void> confirmSignUp(String email, String code) async {
  await Amplify.Auth.confirmSignUp(
    username: email,
    confirmationCode: code,
  );
}
```

كيف يتحقق الكود إن المستخدم مسجل دخوله

عندنا دالة مهمة في GENZ، في `aws_storage.dart` في ملف


```
// requireSignedIn - يتحقق إن المستخدم مسجل دخوله قبل أي عملية
static Future<void> requireSignedIn() async {
  final session = await Amplify.Auth.fetchAuthSession();
  if (!session.isSignedIn) {
    throw Exception('المستخدم غير مسجل الدخول');
  }
}
```

زي الحارس على الباب — **requireSignedIn()** كل عملية في التطبيق بتبدأ بـ

الدخول بالإيميل، باسورد 8 أحرف، في GENZ: AWS هو نظام تسجيل الدخول في Cognito: ملخص القسم الرابع
للمصاحيات Identity Pool، للمستخدمين User Pool. تأكيد الإيميل إجباري.

5. كيف نبعت ونستقبل البيانات — API (AppSync + GraphQL) الـ 📡

API؟ ما هو الـ

كـ "نادل في مطعم API تخيل الـ

- أنت (التطبيق) بتطلب أكلة (بيانات)
- بيروح للمطبخ (قاعدة البيانات) (API) النادل
- ويرجع بالأكلة (البيانات) ليك

AWS AppSync؟ ما هو

AppSync يعمل GraphQL APIs خاصة بالـ AWS هو خدمة

- استقبال طلبات البيانات
- (مع Cognito) التحقق من الهوية
- (DynamoDB) التواصل مع قاعدة البيانات
- إرجاع البيانات للتطبيق
- دعم Real-time (Subscriptions)

GraphQL؟ ما هو

هي لغة لطلب البيانات. بدل ما تطلب "هاتلي كل بيانات المستخدم"، بتقول بالضبط "هاتلي الاسم والإيميل GraphQL
بس."

REST API (القديم):

```
GET /users/123
→ يرجع كل البيانات حتى اللي مش محتاجها
{id, name, email, phone, address, createdAt, updatedAt, ...}
```

GraphQL (الحديث):

```
query {
  getUser(id: "123") {
    name
    email
  }
}
```

→ يرجع بس اللي طلبته

```
{name: "Ahmed", email: "ahmed@gmail.com"}
```

أسرع وأكفاً لأن بياخد بيانات أقل من النت

GraphQL أنواع العمليات في

GraphQL Operations

- |
- |— Query → قراءة البيانات (SELECT في SQL)
- |— Mutation → كتابة/تعديل/حذف البيانات (INSERT/UPDATE/DELETE)
- |— Subscription → استقبال البيانات بشكل حي (Real-time)

تلقائياً API بيولّد ال Amplify كيف

دي لكل جدول Operations بيولّد تلقائياً كل ال AppSync، **amplify push** لما بتعمل

مثلاً **Studio** للـ:

```
# Queries (قراءة)
getStudio(id: ID!): Studio
listStudios(filter: ...): StudioConnection

# Mutations (كتابة)
createStudio(input: CreateStudioInput!): Studio
updateStudio(input: UpdateStudioInput!): Studio
deleteStudio(input: DeleteStudioInput!): Studio

# Subscriptions (حي)
onCreateStudio: Studio
onUpdateStudio: Studio
onDeleteStudio: Studio
```

!كل ده بدون ما تكتب سطر كود

Flutter في API كيف بنستخدم الـ

AppSync بتسهّل الكلام مع **amplify_api** اسمها package بيوفر Amplify، Flutter في

طريقتين للاستخدام:

الأسهل: Model-based الطريقة 1

```
// جديد Studio إنشاء
final studio = Studio(
  name: "استوديو النجوم",
  type: "تسجيل صوتي",
  pricePerHour: 150,
);

final response = await Amplify.API
  .mutate(request: ModelMutations.create(studio))
  .response;
```

model تلقائياً من ال GraphQL query يبيني ال Amplify.

للحالات المعقدة: Raw GraphQL الطريقة 2

```
const query = '''
  query GetStudio($id: ID!) {
    getStudio(id: $id) {
      id name type pricePerHour
    }
  }
''';

final response = await Amplify.API.query(
  request: GraphQLRequest<String>(
    document: query,
    variables: {'id': 'studio-001'},
  ),
).response;
```

طريقة حديثة لطلب البيانات — GraphQL = GraphQL API لل AWS خدمة = AppSync: ملخص القسم الخامس
(حي) Subscription, (كتابة) Mutation, (قراءة) Query: يتطلب بس اللي محتاجه. 3 أنواع عمليات

6. تخزين الصور — Storage (S3) ال

ما هو Amazon S3؟

S3 = Simple Storage Service — تخيله كـ "USB مكان أي ملف وتقدر توصله من أي مكان USB" تخيله كـ في العالم.

بنستخدمه لـ GENZ في:

- صور الاستوديوهات
- صور المستخدمين (Profile pictures)

ال Bucket

كاملة بتأجرها hard drive هو "ال فولدر الرئيسي". زي S3 في **Bucket** ال

```
S3 Bucket: genz-storage-bucket
├── public/
│   └── studios/
│       ├── studio-001.jpg
│       └── studio-002.jpg
└── private/
    └── users/
        ├── ahmed@gmail.com/profile.jpg
        └── sara@gmail.com/profile.jpg
```

Flutter في S3 رفع صورة على

```
// رفع صورة استوديو
static Future<String?> uploadStudioImage(File imageFile, String studioId) async {
  try {
    await requireSignedIn();

    final key = 'studios/$studioId.jpg'; // المسار في S3

    final result = await Amplify.Storage.uploadFile(
      localFile: AWSFile.fromPath(imageFile.path),
      key: key,
      options: const StorageUploadFileOptions(
        accessLevel: StorageAccessLevel.guest, // عام - الكل يشوفه
      ),
    ).result;

    return result.uploadedItem.key; // يرجع المسار
  } catch (e) {
    safePrint('خطأ في الرفع: $e');
    return null;
  }
}
```

الصورة URL الحصول على

```
// صورة الاستوديو URL جلب
static Future<String?> getStudioImageUrl(String key) async {
  try {
    final result = await Amplify.Storage.getUrl(
      key: key,
```

```

options: const StorageGetUrlOptions(
  accessLevel: StorageAccessLevel.guest,
  pluginOptions: S3GetUrlPluginOptions(
    expiresIn: Duration(hours: 1), // الرابط صالح ساعة
  ),
),
).result;

return result.url.toString();
} catch (e) {
  return null;
}
}

```

S3 مستويات الوصول في

StorageAccessLevel

```

|
|— guest → الكل يشوفه (حتى بدون دخول) ← للصور العامة
|— protected → بس المستخدم المسجل يشوفه ← للصور الشخصية
|— private → بس صاحبه يشوفه ← للملفات الخاصة جداً

```

في GENZ:

- (الكل يشوفها) **guest**: صور الاستوديوهات
- **protected**: صور المستخدمين

مستويات 3. S3 الفولدر الرئيسي في = Bucket. تخزين الملفات والصور في السحابة = S3: ملخص القسم السادس
 وصول: guest (عام), protected, private.

7. كيف يبدأ التطبيق — main.dart ملف

main.dart ما هو

زي "مفتاح التشغيل" — لما المستخدم يفتح التطبيق، أول كود Flutter هو "أول ما يشتغل" في أي تطبيق **main.dart**. **main.dart** بيتنفذ هو اللي في

بيبدأ في التطبيق Amplify كيف

```

Future<void> _configureAmplify() async {
  final List<AmplifyPluginInterface> plugins = [
    AmplifyAPI(
      options: APIPluginOptions(modelProvider: ModelProvider.instance),
    ),
    AmplifyAuthCognito(),
    AmplifyStorageS3(),
  ];
}

```

```
await Amplify.addPlugins(plugins);
await Amplify.configure(amplifyconfig);
}
```

شرح كل سطر:

Future<void> _configureAmplify() async

Future<void> = الدالة دي بتاخذ "وقت" عشان تشتغل
 = void تعني مش بترجع قيمة
 _configureAmplify = اسم الدالة (private الـ _ تعني)
 async = جوا await هتستخدم

final List<AmplifyPluginInterface> plugins = [...]

Amplify بيضيف قدرة ل plugin كل (الإضافات) Plugins من الـ (List) بنعمل قائمة

AmplifyAPI(options: APIPluginOptions(modelProvider: ModelProvider.instance))

AmplifyAPI = AppSync الإضافة اللي بتربطنا بـ
 APIPluginOptions = إعدادات الإضافة
 modelProvider = "بتاعتنا models اعرف الـ Amplify" بيقول لـ
 ModelProvider.instance = models (Studio, BookingRequest, ...) بيحجب قائمة كل الـ

لماذا ModelProvider مهم؟

يعني بدله مش هتقدر تكتب Dart objects لـ AppSync مش هيعرف إزاي يحول البيانات الجاية من Amplify، بدونها

```
final studio = response.data; // null! هيكون
```

AmplifyAuthCognito()

AmplifyAuthCognito = Cognito الإضافة اللي بتربطنا بـ
 = تسجيل الدخول والخروج
 = بدونها: مش هتقدر تسجل دخول

AmplifyStorageS3()

AmplifyStorageS3 = S3 الإضافة التي بتربطنا بـ
 رفع وتحميل الملفات والصور
 بدونها: مش هتقدر ترفع صور

await Amplify.addPlugins(plugins)

addPlugins = "أضف هذه الإضافات لـ"
 plugin = بيسجل كل
 configure = لازم تنفذ قبل

await Amplify.configure(amplifyconfig)

configure = "اعد نفسك بهذه المعلومات"
 amplifyconfig = ملف الإعدادات المولد تلقائياً
 = lib/amplifyconfiguration.dart موجود في
 Keys والـ API يحتوي على عناوين الـ

ملف amplifyconfiguration.dart

مثاله. **amplify push** هذا الملف بيتولد تلقائياً بعد

```
const amplifyconfig = '{
  "UserAgent": "aws-amplify-cli/2.0",
  "Version": "1.0",
  "api": {
    "plugins": {
      "awsAPIPlugin": {
        "genz": {
          "endpointType": "GraphQL",
          "endpoint": "https://xxxxxxxxx.appsync-api.us-east-1.amazonaws.com/graphql",
          "region": "us-east-1",
          "authorizationType": "AMAZON_COGNITO_USER_POOLS"
        }
      }
    }
  },
  "auth": {
    "plugins": {
      "awsCognitoAuthPlugin": {
        "UserAgent": "aws-amplify-cli/0.1.0",
        "Version": "0.1.0",
        "IdentityManager": { "Default": {} },

```

```

        "CognitoUserPool": {
            "Default": {
                "PoolId": "us-east-1_XXXXXXXX",
                "AppClientId": "XXXXXXXXXXXXXXXXXXXXXXXXXXXX",
                "Region": "us-east-1"
            }
        }
    },
    "storage": {
        "plugins": {
            "awsS3StoragePlugin": {
                "bucket": "genz-storage-bucket",
                "region": "us-east-1",
                "defaultAccessLevel": "guest"
            }
        }
    }
}'''

```

الترتيب الكامل لبداية التطبيق

```

void main() async {
  // 1. جاهز Flutter تأكد إن
  WidgetsFlutterBinding.ensureInitialized();

  // 2. إعداد Amplify
  await _configureAmplify();

  // 3. تشغيل التطبيق
  runApp(MyApp());
}

```

لماذا هذا الترتيب مهم؟

```

WidgetsFlutterBinding.ensureInitialized()
↓
Flutter مع async code للعمل مع Flutter يجهز محرك
↓
_configureAmplify()
↓
AWS (Cognito + AppSync + S3) يربط التطبيق بـ
↓
runApp(MyApp())
↓
يشغل الواجهة - دلوكتي البيانات جاهزة

```


مش متجهز بعد API هيحصل خطأ لأن الـ `configureAmplify` قبل `runApp` لو شغلت

plugins: بتضيف 3 `_configureAmplify()`. هو أول كود يشتغل في التطبيق `main.dart`: **ملخص القسم السابع**
 Initialize: الترتيب. Amplify. هو ملف الإعدادات المولّد تلقائياً من `amplifyconfig`. API + Auth + Storage.
 Flutter → Configure Amplify → Run App.

8. المحرك الأساسي — `aws_storage.dart` ملف

`aws_storage.dart` ما هو

AWS. بيحتوي على كل العمليات اللي بنتعامل بيها مع — GENZ مركزي في مشروع Dart هو ملف

تخيله كـ "مدير الخدمات" — أي شاشة في التطبيق عايزة بيانات، بتكلم الملف ده

```

في التطبيق Screen أي
    ↓
AWSStorageService (aws_storage.dart)
    ↓
AWS AppSync / S3 / Cognito
    ↓
Screen البيانات ترجع للـ
  
```

الهيكل العام للملف

```

class AWSStorageService {
  // =====
  // Helper Functions
  // =====
  static Future<void> requireSignedIn() async { ... }

  // =====
  // Studio Operations
  // =====
  static Future<List<Studio>> getStudios() async { ... }
  static Future<bool> createStudio(Studio studio) async { ... }

  // =====
  // Booking Operations
  // =====
  static Future<bool> createBooking(BookingRequest booking) async { ... }
  static Future<List<BookingRequest>> getBookings() async { ... }
  static Future<bool> updateBookingStatus(String id, String status) async { ... }

  // =====
  // Notification Operations
  // =====
  static Future<bool> sendNotification({...}) async { ... }
  static Future<List<AppNotification>> getNotifications(String email) async { ... }
  
```

```

}

// =====
// Chat Operations
// =====
static Future<bool> sendMessage({...}) async { ... }
static Stream<ChatMessage> subscribeToChatMessages({...}) { ... }

// =====
// UserProfile Operations
// =====
static Future<UserProfile?> getUserProfile(String email) async { ... }
static Future<bool> setChatEnabled(String email, bool enable) async { ... }
static Future<bool> isChatEnabled(String email) async { ... }
}

```

static لماذا كل الدوال

عشان تستخدمها class من ال object تعني إنك مش محتاج تعمل static الكلمة

```

// object محتاج تعمل - static بدون
final service = AWSStorageService();
await service.sendNotification(...);

// بتكلمها مباشرة - static مع
await AWSStorageService.sendNotification(...);

```

عشان static اخترنا GENZ في

1. أسهل في الاستخدام
2. object مفيش حاجة تخزنها في ال
3. كل الشاشات بتشاركها بشكل أبسط

static بتمر بيه. كل الدوال AWS هو المحرك المركزي — كل عمليات aws_storage.dart: **ملخص القسم الثامن**
عشان سهولة الاستخدام من أي مكان في التطبيق.

9. 🚀 كيف تبعت البيانات (Mutations)

Mutation ما هي ال

: "بتعني" أي عملية بتغير البيانات GraphQL بالعربي تعني "تغيير" أو "طفرة" — وفي **Mutation**

- (Create) إنشاء بيانات جديدة
- (Update) تعديل بيانات موجودة
- (Delete) حذف بيانات

(Create Mutation) مثال 1: إرسال إشعار

```
// إرسال إشعار
static Future<bool> sendNotification({
  required String clientEmail,
  required String title,
  required String body,
  String type = 'info',
}) async {
  await requireSignedIn();
  final notif = AppNotification(
    clientEmail: clientEmail,
    title: title,
    body: body,
    type: type,
    time: DateTime.now().toUtc().toIso8601String(),
    read: false,
  );
  final response = await Amplify.API
    .mutate(request: ModelMutations.create(notif))
    .response;
  return response.errors.isEmpty;
}
```

شرح سطر سطر:

السطر: `static Future<bool> sendNotification({...}) async`

static = object تقدر تستدعيها بدون
 Future<bool> = بعد انتظار (فشل) false أو (نجاح) true بترجع
 required = الباراميتر ده إجباري
 String type = 'info' = القيمة الافتراضية

السطر: `await requireSignedIn()`

قبل أي عملية، تأكد إن المستخدم مسجل دخوله.
 وبتوقف هنا Exception لو مش مسجل - بيرمي

السطر: `final notif = AppNotification(...)`

AppNotification من نوع object بنعمل
 amplify codegen models اتولد تلقائياً من class هذا الـ

السطر: `time: DateTime.now().toUtc().toIso8601String()`

<code>DateTime.now()</code>	= الوقت الحالي على الجهاز
<code>.toUtc()</code>	= (وقت عالمي موحد) UTC حوله لـ
<code>.toIso8601String()</code>	= "15-01-2024T10:30:00.000Z" حوله لنص:

؟. عشان لو في عميل في مصر وموظف في دبي — الوقت هيتحسب صح للاتنين UTC لماذا

السطر: `final response = await Amplify.API.mutate(request: ModelMutations.create(notif)).response`

<code>Amplify.API</code>	= الـ API plugin
<code>.mutate(...)</code>	= (تغيير في البيانات) mutation عملية
<code>ModelMutations.create(notif)</code>	= "insert اعمل object في DynamoDB لهذا الـ"
<code>.response</code>	= انتظر الرد من السيرفر

ما الذي يحدث خلف الكواليس؟

Dart Code

↓

Amplify Library بتلقائياً GraphQL Mutation بيبي

```
mutation CreateAppNotification($input: CreateAppNotificationInput!) {
  createAppNotification(input: $input) {
    id clientEmail title body type time read _version
  }
}
```

↓

AWS AppSync بيبيعه لـ

↓

DynamoDB بيكتبه في AppSync

↓

الجديد id المنشأ مع الـ object بيرجع الـ

السطر: `return response.errors.isEmpty`

<code>errors</code> لو مفيش	→ <code>true</code> (تم الحفظ بنجاح)
<code>errors</code> لو في	→ <code>false</code> (حصل خطأ)

(Update Mutation) مثال 2: تحديث حالة الحجز

```
static Future<bool> updateBookingStatus(String id, String status) async {
  await requireSignedIn();
```

```
// _version الحالي مع booking أولاً: اجيب الـ
final getRequest = ModelQueries.get(BookingRequest.classType,
    BookingRequestModelIdentifier(id: id));
final getResponse = await Amplify.API.query(request: getRequest).response;

if (getResponse.data == null) return false;

final booking = getResponse.data!;

// ثانياً: اعمل نسخة معدلة
final updatedBooking = booking.copyWith(status: status);

// ثالثاً: ابعت التحديث
final updateRequest = ModelMutations.update(updatedBooking);
final updateResponse = await Amplify.API.mutate(request:
updateRequest).response;

return updateResponse.errors.isEmpty;
}
```

(Raw GraphQL) يدوياً **_version** مثال 3: تحديث مع

هذا المثال الأهم والأصعب في المشروع

```
// conflict detection (مطلوب بسبب) _version مع chatEnabled تحديث
const getDoc = 'query GetUserProfile($id: ID!) { getUserProfile(id: $id) { id
_version } }';
const mutDoc = 'mutation UpdateUserProfile($input: UpdateUserProfileInput!) {
updateUserProfile(input: $input) { id chatEnabled _version } }';

// الحالي _version الخطوة 1: اجيب الـ
final getResp = await Amplify.API.query(
    request: GraphQLRequest<String>(document: getDoc, variables: {'id':
profile.id}),
).response;

// من النتيجة _version الخطوة 2: استخرج الـ
int version = 1;
final raw = getResp.data ?? '{}';
final idx = raw.indexOf('"_version":');
if (idx >= 0) {
    final sub = raw.substring(idx + 11);
    final end = sub.indexOf(RegExp(r'[ ,]'));
    version = int.tryParse(sub.substring(0, end).trim()) ?? 1;
}

// _version الخطوة 3: ابعت التحديث مع الـ
await Amplify.API.mutate(
    request: GraphQLRequest<String>(
        document: mutDoc,
```

```
variables: {'input': {'id': profile.id, 'chatEnabled': enable, '_version':
version}},
),
).response;
```

وليه مهم جداً **_version** شرح ال

تخيل إنك وصديقك بتعدّلوا نفس الملف في نفس الوقت

عدّلت → حفظت → (version 1) أنت: فتحت الملف
عدّلت → حفظت → (version 1) صديقك: فتحت الملف

مشكلة! إيه اللي اتحفظ؟ تعديلك أنت ولا تعديل صديقك؟

بيحل المشكلة دي Amplify في **Conflict Detection** ال

رقم **_version** عنده DynamoDB في record كل
لما حد يعدّل:

- الحالي مع التعديل **_version** لازم يبعث ال -
- conflict = مختلف DB الحالي في ال **_version** لو ال -
- AppSync يرفض التحديث ويرجع خطأ

خطوة بخطوة:

```
DB = 3 في ال _version ال
↓
Step 1: query getUserProfile → يرجع {id: "xxx", _version: 3}
↓
Step 2: version = 3 استخرج
↓
Step 3: mutation updateUserProfile(input: {id: "xxx", chatEnabled: true, _version: 3})
↓
AppSync (3) DB في ال _version المبعوث (3) = ال _version يقارن: هل ال
↓ نعم
لـ 4 _version ويرفع ال record يحدّث ال
↓ لا
(Conflict!) يرفض التحديث
```

Model-based هنا بدل ال Raw GraphQL لماذا نعمل

في بعض الحالات أو بيتسبب في **_version** أحياناً بيتجاهل ال **ModelMutations.update()** Model-based لأن ال
بيعطيك تحكم كامل Raw GraphQL تعارضات. ال

version: شرح كود استخراج ال

```

final raw = getResp.data ?? '{}';
// raw = '{"getUserProfile":{"id":"abc","_version":3}}'

final idx = raw.indexOf('"_version:');
// idx = string في الـ '"_version:" موقع النص

if (idx >= 0) {
  final sub = raw.substring(idx + 11);
  // idx + 11 = مباشرة '"_version:" نقطة بعد
  // sub = '3}}'

  final end = sub.indexOf(RegExp(r'[,}]]'));
  // بدور على أول فاصلة أو قوس إغلاق
  // end = 1 (موقع '}')

  version = int.tryParse(sub.substring(0, end).trim()) ?? 1;
  // sub.substring(0, 1) = '3'
  // int.tryParse('3') = 3
  // version = 3
}

```

GENZ في Mutations جدول أنواع الـ

Mutation الـ	الوصف	متى بتستخدمها
<code>ModelMutations.create(obj)</code>	جديد record إنشاء	حجز جديد، إشعار جديد، رسالة جديدة
<code>ModelMutations.update(obj)</code>	record تعديل موجود	تعديل حالة الحجز
<code>ModelMutations.delete(obj)</code>	record حذف	حذف إشعار
<code>GraphQLRequest<String>(document: mutDoc)</code>	يدوي Mutation	زي (ال) لما محتاج تحكم كامل _version)

Mutation = أي عملية بتغيّر البيانات (create/update/delete). ملخص القسم التاسع

— conflict detection مهم جداً مع الـ `_version`. record أسهل طريقة لإنشاء `ModelMutations.create()` update دائماً اجلبه وابعته مع أي

10. (Queries) كيف تستقبل البيانات

Query؟ ما هي الـ

تعني "استعلام" — طلب قراءة بيانات من قاعدة البيانات بدون ما تغيّر فيها حاجة **Query**.

مثال 1: جلب كل الاستوديوهات

```
static Future<List<Studio>> getStudios() async {
  await requireSignedIn();

  final request = ModelQueries.list(Studio.classType);
  final response = await Amplify.API.query(request: request).response;

  return response.data?.items.whereType<Studio>().toList() ?? [];
}
```

شرح:

```
ModelQueries.list(Studio.classType)
// تلقائياً query يبني هذا الـ:
// query ListStudios {
//   listStudios {
//     items { id name type pricePerHour description image available }
//   }
// }

response.data?.items
// هي قائمة الاستوديوهات items الـ

.whereType<Studio>()
// null values فلتّر - بيشيل أي

.toList()
// List<Studio> يحولها لـ

?? []
// ارجع قائمة فاضية → null لو البيانات
```

(Raw GraphQL Query) مثال 2: جلب إشعارات عميل معين

```
// جلب إشعارات العميل
const doc = '''
  query ListByEmail($email: String!) {
    appNotificationsByClientEmail(clientEmail: $email) {
      items { id clientEmail title body type time read _version }
    }
  }
''';

final resp = await Amplify.API.query(
  request: GraphQLRequest<String>(document: doc, variables: {'email': email}),
).response;
```

هنا؟ Raw GraphQL لماذا

ال Schema في ال `@index` موجود بسبب ال **custom query** هو `appNotificationsByClientEmail` لأن `ModelQueries.list()` بس إنا عايزين إشعارات عميل معين بس **بيجيب كل الإشعارات** — بس إنا عايزين إشعارات عميل معين بس `ModelQueries.list()`.

Query: شرح ال

```
query ListByEmail($email: String!) {
  # ال @index المولد من query اسم ال
  appNotificationsByClientEmail(clientEmail: $email) {
    items {
      id
      clientEmail
      title
      body
      type
      time
      read
      _version    # conflict detection مهم لل
    }
  }
}
```

Variables:

```
variables: {'email': 'ahmed@gmail.com'}
// query في ال $email بيستبدل
```

معالجة الاستجابة:

```
final raw = resp.data ?? '{}';
// raw = '{"appNotificationsByClientEmail":{"items":[...]}}'

// يعطيك قائمة الإشعارات JSON parsing ال
final decoded = jsonDecode(raw);
final items = decoded['appNotificationsByClientEmail']['items'] as List;

final notifications = items.map((item) => AppNotification(
  id: item['id'],
  clientEmail: item['clientEmail'],
  title: item['title'],
  body: item['body'],
  type: item['type'],
  time: item['time'],
  read: item['read'] ?? false,
)).toList();
```

مثال 3: جلب ملف مستخدم معين

```

static Future<UserProfile?> getUserProfile(String email) async {
  await requireSignedIn();

  const doc = '''
    query GetByEmail($email: String!) {
      userProfilesByEmail(email: $email) {
        items { id email name image type chatEnabled lastUpdated _version }
      }
    }
  ''';

  final resp = await Amplify.API.query(
    request: GraphQLRequest<String>(document: doc, variables: {'email': email}),
  ).response;

  // معالجة النتيجة
  if (resp.data == null) return null;

  final raw = resp.data!;
  // ... parsing السابق كالمثال

  return userProfile;
}

```

Query Types الفرق بين

ModelQueries.list()	→ من جدول records كل الـ
ModelQueries.get(id)	→ record واحد بالـ id
GraphQLRequest (custom)	→ query مخصص (مثل @index queries)

Query مع Filter

index: لو عايز تفلتر النتائج بدون استخدام الـ

```

// فقط Pending جلب الحجوزات بحالة
final request = ModelQueries.list(
  BookingRequest.classType,
  where: BookingRequest.STATUS.eq('Pending'),
);

```

أسرع بكثير @index لأنه يجيب كل البيانات ويفلترها بعدين. الـ @index **ملاحظة:** ده أبطأ من الـ

Queries معالجة الأخطاء في الـ

```

try {
    final response = await Amplify.API.query(request: request).response;

    if (response.errors.isNotEmpty) {
        // من الـ errors في GraphQL
        for (final error in response.errors) {
            safePrint('GraphQL Error: ${error.message}');
        }
        return [];
    }

    return response.data?.items.whereType<Studio>().toList() ?? [];
} on ApiException catch (e) {
    // خطأ في الشبكة أو الـ API
    safePrint('API Exception: ${e.message}');
    return [];
} catch (e) {
    // أي خطأ ثاني
    safePrint('Unknown error: $e');
    return [];
}

```

أسهل طريقة لجلب كل `ModelQueries.list()`. قراءة بيانات بدون تعديل Query = ملخص القسم العاشر
أسرع للبحث على حقل معين. دائماً تعامل مع `appNotificationsByClientEmail` (زي `@index queries` البيانات). الـ
errors.

11. ⚡ الـ Real-Time (Subscriptions)

Real-Time ما هو الـ

عشان تشوف رده؟ طبعاً لا! الرد ببيجيك تلقائياً. ده هو refresh تخيل إنك بتبعت رسالة لصديقك على واتساب. لازم تضغط
Real-time.

في التطبيقات العادية

"التطبيق يسأل السيرفر كل 5 ثواني: "في بيانات جديدة؟
السيرفر: لأ، لأ، لأ، آه (رسالة وصلت)

مع Real-time Subscriptions:

التطبيق فاتح "قناة اتصال" مفتوحة مع السيرفر
السيرفر: "وصلت رسالة جديدة!" → يبعث على طول

Subscriptions في Amplify كيف تعمل الـ

WebSocket Connection

التطبيق ← → AWS AppSync

هو نوع خاص من الاتصال — بدل الاتصال والقطع مع كل طلب، يفتح اتصال مستمر **WebSocket** الـ.

مثال: الاستماع لرسائل شات جديدة

```
// الاستماع لرسائل شات جديدة
static Stream<ChatMessage> subscribeToChatMessages({String? clientEmail}) {
  final controller = StreamController<ChatMessage>.broadcast();

  final sub = Amplify.API.subscribe(
    ModelSubscriptions.onCreate(ChatMessage.classType),
    onEstablished: () => safePrint('🔥 ChatMessages established'),
  ).listen(
    (event) {
      if (event.data == null) return;
      if (clientEmail != null &&
          event.data!.clientEmail.toLowerCase() != clientEmail.toLowerCase())
        return;
      controller.add(event.data!);
    },
    onError: (e) => safePrint('ChatMessages sub error: $e'),
  );

  controller.onCancel = () => sub.cancel();
  return controller.stream;
}
```

شرح كل جزء:

Stream<ChatMessage>

Stream = تدفق بيانات مستمر (زي نهر)
 بدل ما يرجع قيمة واحدة، بيرجع قيم متعددة على مر الوقت
 ChatMessage = نوع البيانات اللي في الـ Stream

الفرق:

Future<ChatMessage> = انتظر وهيچيك رسالة واحدة
 Stream<ChatMessage> = ابقى جاهز هتجيك رسائل متعددة على مر الوقت

StreamController<ChatMessage>.broadcast()

StreamController = "Stream مراقب" بيتحكم في الـ
 .broadcast() = يسمح لأكثر من مستمع واحد
 = واحد يقدر يستمع widget بدونه: بس

Amplify.API.subscribe(ModelSubscriptions.onCreate(ChatMessage.classType))

subscribe = اشترك "في الأحداث"
 ModelSubscriptions.onCreate = جديد يتعمل ChatMessage اسمعني لما
 ChatMessage.classType = اللي بنستمعه model نوع الـ

تلقائياً GraphQL هيوّلد هذا Subscription الـ

```
subscription OnCreateChatMessage {
  onCreateChatMessage {
    id senderName senderEmail clientEmail text time messageType parentId
  }
}
```

onEstablished: () => safePrint('👉 ChatMessages established')

onEstablished = callback بيتنفذ لما الاتصال يتأسس
 = "بيقولك" الاشتراك شغال دلوقتي

(event) { ... }.listen(...)

listen = "ابدأ الاستماع"
 event = الحدث الجديد اللي وصل
 event.data = الـ ChatMessage الجديد

```
if (clientEmail != null && event.data!.clientEmail.toLowerCase() !=
clientEmail.toLowerCase()) return;
```

هذا الكود مهم جداً

مش بس رسائل عميل معين — DB بيستمع لكل رسائل الشات في الـ Subscription المشكلة: الـ

الحل: نفلتر في الكود:

لو الرسالة مش بتاعة العميل ده → تجاهلها
Stream لو الرسالة بتاعته → أضفها للـ

عشان نتجنب مشاكل الحروف الكبيرة والصغيرة (`.toLowerCase()`).

`controller.onCancel = () => sub.cancel()`

يتحذف (مثلاً لما المستخدم يقفل الشاشة) widget لما الـ
→ يتلغى Stream الـ
→ sub.cancel() يوقف الـ WebSocket connection
→ memory leak مش هيضيع باند ومش هيحصل

Subscriptions أنواع الـ

```
// جديد يتعمل record استمع لما
ModelSubscriptions.onCreate(ChatMessage.classType)

// يتعدل record استمع لما
ModelSubscriptions.onUpdate(ChatMessage.classType)

// يتحذف record استمع لما
ModelSubscriptions.onDelete(ChatMessage.classType)
```

Widget في الـ Stream كيف تستخدم الـ

```
class ChatScreen extends StatefulWidget { ... }

class _ChatScreenState extends State<ChatScreen> {
  StreamSubscription<ChatMessage>? _subscription;
  List<ChatMessage> _messages = [];

  @override
  void initState() {
    super.initState();
    _startListening();
  }

  void _startListening() {
    _subscription = AWSStorageService
      .subscribeToChatMessages(clientEmail: widget.clientEmail)
      .listen((newMessage) {
        setState(() {
```

```

        _messages.add(newMessage);
    });
}

@override
void dispose() {
    _subscription?.cancel(); // مهم: وقف الاستماع لما الشاشة تتقفل
    super.dispose();
}
}

```

وحلولها Subscriptions مشاكل الـ

المشكلة	السبب	الحل
مش بتشتغل الـ Subscription	network مشكلة في الـ	fallback كـ Polling استخدم
بيانات تيجي للكل مش لعمل معين	ييسم للكل الـ Subscription	فلتر في الكود
Memory leak	cancel ننسى نعمل	دائماً <code>dispose()</code>
تأخر في الاتصال	Websocket للتأسيس وقت	<code>onEstablished</code> callback

ملخص القسم الحادي عشر: Subscription = refresh استقبال بيانات حية بدون WebSocket connection يعمل. لما الشاشة تتقفل cancel تدفق مستمر من البيانات. مهم جداً: عمل = Dart في AppSync. Stream مفتوح مع

12. 🛎 نظام الإشعارات بالتفصيل

GENZ كيف يعمل نظام الإشعارات في

مبني **in-app notifications** ده نظام (Firebase FCM) Push Notifications مش GENZ نظام الإشعارات في DynamoDB + GraphQL Subscriptions.

الفكرة:

```

حدث ما (حجز، قبول، رفض...)
↓
DynamoDB في createAppNotification كود يعمل
↓
على الجهاز الآخر بيسمعه Subscription
↓
إشعار يظهر في التطبيق

```

الـ Sentinel Key: `'__employees__'`

!هذا من أذكى أجزاء النظام

المشكلة: كيف نبعت إشعار لـ "كل الموظفين"؟ مش عارفين إيميلات الموظفين دائماً

الحل: `'__employees__'` اتفقنا على كلمة سرية

```
// '__employees__' لما عميل يحجز، نبعت إشعار لـ
await AWSStorageService.sendNotification(
  clientEmail: '__employees__', // sentinel key
  title: 'احجز جديد!',
  body: 'العميل $clientName طلب حجز استوديو $studioName',
  type: 'Pending',
);
```

```
// '__employees__' الموظف يستمع للإشعارات على
AWSStorageService.subscribeToNotifications(
  clientEmail: '__employees__', // sentinel key
);
```

بالتالي:

- `__employees__` كل الموظفين يستمعوا على
- يوصل لكل الموظفين `__employees__` أي إشعار يتبعته على

GENZ تدفق الإشعارات الكامل في

سيناريو: عميل يحجز

- العميل يضغط "احجز" في التطبيق
↓
- يتم عمل BookingRequest في DynamoDB (status: "Pending")
↓
- يتم عمل AppNotification:
 - clientEmail: "__employees__"
 - title: "احجز جديد"
 - type: "Pending"
 ↓
- "__employees__" شغل على Subscription الموظف عنده
↓
- الإشعار يوصله فوراً في تطبيق الموظف
↓
- الموظف يضغط "قبول" أو "رفض"
↓
- يتم تحديث status الـ BookingRequest
↓
- يتم عمل AppNotification:
 - clientEmail: "ahmed@gmail.com" (إيميل العميل)
 - title: "تم قبول/رفض حجزك"
 - type: "Approved" / "Rejected"
 ↓

شغل على إيميله Subscription العميل عنده

↓

الإشعار يوصله فوراً

سيناريو فتح الشات

1. الموظف يضغط "فتح الشات" للعميل

↓

2. UserProfile.chatEnabled = true بيتحدث

↓

3. AppNotification: بيتعمل

- clientEmail: "ahmed@gmail.com"

- title: "تم فتح الشات"

- type: "chat_opened"

↓

4. العميل يستلم الإشعار

↓

5. التطبيق يعرف إن الشات مفتوح → يفتح واجهة الشات

كود إرسال الإشعارات

```
// إرسال إشعار للموظفين (عند الحجز الجديد)
static Future<void> notifyEmployeesNewBooking({
  required String clientName,
  required String studioName,
  required String date,
}) async {
  await sendNotification(
    clientEmail: '__employees__',
    title: 'حجز جديد من $clientName',
    body: 'التاريخ: $date\nاستوديو: $studioName',
    type: 'Pending',
  );
}

// إرسال إشعار للعميل (عند القبول)
static Future<void> notifyClientApproved({
  required String clientEmail,
  required String studioName,
}) async {
  await sendNotification(
    clientEmail: clientEmail,
    title: 'تم قبول حجزك',
    body: '$studioName مبروك! تم قبول حجز',
    type: 'Approved',
  );
}

// إرسال إشعار للعميل (عند الرفض)
```

```
static Future<void> notifyClientRejected({
  required String clientEmail,
  required String reason,
}) async {
  await sendNotification(
    clientEmail: clientEmail,
    title: 'تم رفض حجزك',
    body: 'عذراً، تم رفض الحجز. السبب: $reason',
    type: 'Rejected',
  );
}

// إرسال إشعار للعميل (عند فتح الشات)
static Future<void> notifyClientChatOpened({
  required String clientEmail,
}) async {
  await sendNotification(
    clientEmail: clientEmail,
    title: 'تم فتح الشات',
    body: 'يمكنك الآن التحدث مع فريقنا مباشرة',
    type: 'chat_opened',
  );
}
```

جدول أنواع الإشعارات

المعنى	لمن	من يبعثه	النوع (type)
حجز جديد ينتظر المراجعة	__employees__	العميل	Pending
تم قبول الحجز	إيميل العميل	الموظف	Approved
تم رفض الحجز	إيميل العميل	الموظف	Rejected
تم فتح خاصية الشات	إيميل العميل	الموظف	chat_opened

"قراءة الإشعارات وتعليمها كـ"مقروءة"

```
// تعليم الإشعار كمقروء
static Future<bool> markNotificationAsRead(AppNotification notif) async {
  await requireSignedIn();

  // _version نحتاج الـ
  final updatedNotif = notif.copyWith(read: true);

  final request = ModelMutations.update(updatedNotif);
  final response = await Amplify.API.mutate(request: request).response;

  return response.errors.isEmpty;
}

// حذف إشعار
```

```
static Future<bool> deleteNotification(AppNotification notif) async {
  await requireSignedIn();

  final request = ModelMutations.delete(notif);
  final response = await Amplify.API.mutate(request: request).response;

  return response.errors.isEmpty;
}
```

شاشة الإشعارات (notifications_screen.dart)

```
class NotificationsScreen extends StatefulWidget { ... }

class _NotificationsScreenState extends State<NotificationsScreen> {
  List<AppNotification> _notifications = [];
  StreamSubscription? _sub;

  @override
  void initState() {
    super.initState();
    _loadNotifications();
    _subscribeToNewNotifications();
  }

  // جلب الإشعارات الموجودة
  Future<void> _loadNotifications() async {
    final notifs = await AWSStorageService.getNotifications(widget.userEmail);
    setState(() => _notifications = notifs);
  }

  // الاستماع للإشعارات الجديدة
  void _subscribeToNewNotifications() {
    _sub = AWSStorageService
      .subscribeToNotifications(clientEmail: widget.userEmail)
      .listen((newNotif) {
        setState(() => _notifications.insert(0, newNotif));
      });
  }

  @override
  void dispose() {
    _sub?.cancel();
    super.dispose();
  }
}
```

GENZ + Subscriptions مبنی على DynamoDB ملخص القسم الثاني عشر: نظام الإشعارات في `__employees__` هو sentinel key أنواع إشعارات 4 لكل الموظفين. Pending, Approved, Rejected, chat_opened. GraphQL Subscriptions عن طريق real-time الإشعارات بتوصل.

13. 🗨️ نظام الشات بالتفصيل

GENZ كيف يعمل نظام الشات في

هو محادثة بين عميل واحد وفريق الموظفين GENZ نظام الشات في

الخصائص:

- الشات مش متاح لكل العملاء — لازم الموظف "يفتحة" للعميل أولاً
- الموظف بيرد على العملاء من شاشة واحدة
- real-time الرسائل بتيجي

شرط "chatEnabled"

```
chatEnabled = false    → العميل مش يقدر يشوف أو يفتح الشات
chatEnabled = true     → الشات متاح للعميل
```

```
// التحقق من حالة الشات قبل الإرسال
static Future<bool> isChatEnabled(String email) async {
  try {
    await requireSignedIn();

    final profile = await getUserProfile(email);
    return profile?.chatEnabled ?? false;

  } catch (e) {
    return false;
  }
}
```

إرسال رسالة

```
static Future<bool> sendMessage({
  required String clientEmail,
  required String text,
  required String senderName,
  required String senderEmail,
  String messageType = 'text',
  String? parentId,
}) async {
  await requireSignedIn();

  final message = ChatMessage(
    senderName: senderName,
    senderEmail: senderEmail,
    clientEmail: clientEmail, // إيميل العميل صاحب المحادثة
```

```

    text: text,
    time: DateTime.now().toUtc().toIso8601String(),
    messageType: messageType,
    parentId: parentId, // لو رد على رسالة ثانية
);

final response = await Amplify.API
    .mutate(request: ModelMutations.create(message))
    .response;

return response.errors.isEmpty;
}

```

الاستماع للرسائل الجديدة

```

// الاستماع لرسائل شات جديدة
static Stream<ChatMessage> subscribeToChatMessages({String? clientEmail}) {
    final controller = StreamController<ChatMessage>.broadcast();

    final sub = Amplify.API.subscribe(
        ModelSubscriptions.onCreate(ChatMessage.classType),
        onEstablished: () => safePrint('🔥 ChatMessages established'),
    ).listen(
        (event) {
            if (event.data == null) return;
            if (clientEmail != null &&
                event.data!.clientEmail.toLowerCase() != clientEmail.toLowerCase())
            return;
            controller.add(event.data!);
        },
        onError: (e) => safePrint('ChatMessages sub error: $e'),
    );

    controller.onCancel = () => sub.cancel();
    return controller.stream;
}

```

مخصصة؟ subscription بدل ما نعمل clientEmail لماذا نفلتر على

الشائع هو workaround بشكل مباشر في كل الإصدارات. الـ filter parameters مع subscription مش بيدعم Amplify

1. اشترك في كل الرسائل الجديدة.
2. فلتر في الكود على اللي يخصك.

```

// فلتر الرسائل
if (clientEmail != null &&
    event.data!.clientEmail.toLowerCase() != clientEmail.toLowerCase()) {

```

```
return; // تجاهل هذه الرسالة
}
```

كيف الموظف يشوف كل المحادثات

```
// جلب آخر رسالة لكل عميل
static Future<Map<String, ChatMessage>> getLastMessagePerClient() async {
  await requireSignedIn();

  const doc = '''
    query ListAllMessages {
      listChatMessages(limit: 1000) {
        items { id senderName senderEmail clientEmail text time messageType }
      }
    }
  ''';

  final resp = await Amplify.API.query(
    request: GraphQLRequest<String>(document: doc),
  ).response;

  // فرز الرسائل بالتاريخ وأخذ آخر رسالة لكل عميل
  final Map<String, ChatMessage> lastMessages = {};

  // ... processing code

  return lastMessages;
}
```

Flutter شاشة الشات في

```
class ChatScreen extends StatefulWidget {
  final String clientEmail;
  final String currentUserEmail;
  final bool isEmployee;

  const ChatScreen({
    required this.clientEmail,
    required this.currentUserEmail,
    required this.isEmployee,
  });
}

class _ChatScreenState extends State<ChatScreen> {
  List<ChatMessage> _messages = [];
  StreamSubscription<ChatMessage>? _subscription;
  final TextEditingController _textController = TextEditingController();

  @override
```

```

void initState() {
  super.initState();
  _loadMessages();
  _startRealTimeListening();
}

// تحميل الرسائل القديمة
Future<void> _loadMessages() async {
  const doc = '''
    query GetMessages($email: String!) {
      chatMessagesByClientEmail(clientEmail: $email) {
        items { id senderName senderEmail clientEmail text time messageType
parentId }
      }
    }
  ''';

  final resp = await Amplify.API.query(
    request: GraphQLRequest<String>(
      document: doc,
      variables: {'email': widget.clientEmail},
    ),
  ).response;

  // ... parse and setState
}

// الاستماع للرسائل الجديدة
void _startRealTimeListening() {
  _subscription = AWSStorageService
    .subscribeToChatMessages(clientEmail: widget.clientEmail)
    .listen((newMessage) {
      if (mounted) {
        setState(() => _messages.add(newMessage));
        _scrollToBottom();
      }
    });
}

// إرسال رسالة
Future<void> _sendMessage() async {
  final text = _textController.text.trim();
  if (text.isEmpty) return;

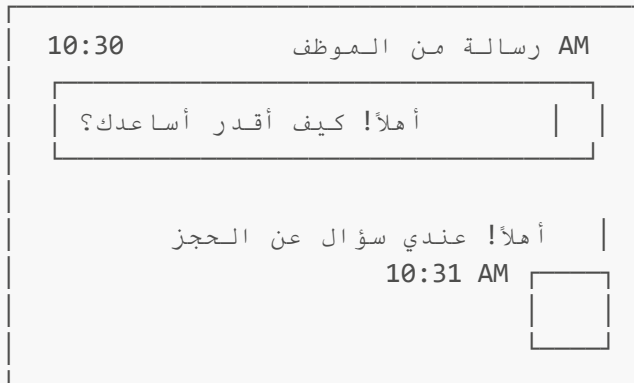
  _textController.clear();

  await AWSStorageService.sendMessage(
    clientEmail: widget.clientEmail,
    text: text,
    senderName: widget.isEmployee ? 'أنا' : 'الموظف',
    senderEmail: widget.currentUserEmail,
  );
}

```

```
@override
void dispose() {
  _subscription?.cancel();
  _textController.dispose();
  super.dispose();
}
}
```

UI هيكل الرسالة في ال



الرسالة من اليسار = موظف الرسالة من اليمين = عميل (أنا)

```
// تحديد موقع الرسالة في ال
bool isMyMessage = message.senderEmail == widget.currentUserEmail;

Align(
  alignment: isMyMessage ? Alignment.centerRight : Alignment.centerLeft,
  child: Container(
    color: isMyMessage ? Colors.blue : Colors.grey,
    child: Text(message.text ?? ''),
  ),
)
```

فتح وإغلاق الشات (من جهة الموظف)

```
// فتح الشات لعميل
static Future<bool> setChatEnabled(String email, bool enable) async {
  await requireSignedIn();

  // الحالي profile الخطوة 1: جيب ال
  final profile = await getUserProfile(email);
  if (profile == null) return false;

  // الخطوة 2: جيب ال _version
  const getDoc = 'query GetUserProfile($id: ID!) { getUserProfile(id: $id) { id
```



```

    _version } }';
    final getResp = await Amplify.API.query(
      request: GraphQLRequest<String>(
        document: getDoc,
        variables: {'id': profile.id},
      ),
    ).response;

    int version = 1;
    final raw = getResp.data ?? '{}';
    final idx = raw.indexOf('"_version":');
    if (idx >= 0) {
      final sub = raw.substring(idx + 11);
      final end = sub.indexOf(RegExp(r'[,}]'));
      version = int.tryParse(sub.substring(0, end).trim()) ?? 1;
    }

    // _version مع الـ chatEnabled الخطوة 3: حدّث
    const mutDoc = '''
      mutation UpdateUserProfile($input: UpdateUserProfileInput!) {
        updateUserProfile(input: $input) {
          id chatEnabled _version
        }
      }
    ''';

    final mutResp = await Amplify.API.mutate(
      request: GraphQLRequest<String>(
        document: mutDoc,
        variables: {
          'input': {
            'id': profile.id,
            'chatEnabled': enable,
            '_version': version,
          },
        },
      ),
    ).response;

    if (mutResp.errors.isEmpty && enable) {
      // لو فتحنا الشات، ابعت إشعار للعميل
      await sendNotification(
        clientEmail: email,
        title: 'تم فتح الشات',
        body: 'يمكنك الآن التواصل معنا مباشرة',
        type: 'chat_opened',
      );
    }

    return mutResp.errors.isEmpty;
  }
}

```

يُتحكم UserProfile في `chatEnabled`. بين عميل وفريق الموظفين GENZ ملخص القسم الثالث عشر: الشات في الموظف يفتح الشات → إشعار يوصل Subscriptions عن طريق real-time في الوصول للشات. الرسائل بتوصل للعميل.

14. Polling — البديل لما الـ

ما هو Polling؟

أنت بتسأله كل فترة: (Subscriptions) بالعربي تعني "الاستطلاع الدوري" — يعني بدل ما تستنى السيرفر بيعتلك **Polling** في جديد؟

Subscriptions:	Polling:
السيرفر → التطبيق (كل 5 ثواني)	التطبيق → السيرفر
(push)	(pull)

ما Polling متى نستخدم

ما شغالة في Subscriptions:

- X WebSocket شبكات معينة بتبلوك
- X نقطة ضعيفة في الاتصال
- X بعض البيئات المقيدة

fallback كـ Polling → في الحالات دي

للتحقق من حالة الشات Polling: مثال

```
// fallback كل 5 ثواني كـ polling
_chatPollingTimer = Timer.periodic(const Duration(seconds: 5), (_) async {
  if (!mounted) return;
  final enabled = await AWSStorageService.isChatEnabled(email);
  // process result...
});
```

شرح كل سطر:

`_chatPollingTimer = Timer.periodic(...)`

Timer.periodic = مؤقت بيتكرر كل فترة
عشان نوقفه لاحقاً `_chatPollingTimer` بيخزن في =

const Duration(seconds: 5)

كل كام يتكرر؟ كل 5 ثواني.
AppSync يعني في دقيقة واحدة: 12 طلب لـ

هل ده مش تقيل على السيرفر؟

في حالتنا: عدد المستخدمين المتوقع صغير، وكل طلب بياخد بيانات صغيرة. مقبول

Subscriptions. لو في ملايين مستخدمين: ممكن تزود الوقت لـ 30 ثانية أو تفضّل على

if (!mounted) return;

لسه موجود في الشجرة؟ Widget هل الـ mounted =
:لو المستخدم خرج من الشاشة
mounted = false
if (!mounted) return; → وقف الـ polling
!اتحذف → خطأ widget على setState بدون السطر ده: هيحاول يعمل

final enabled = await AWSStorageService.isChatEnabled(email);

كل 5 ثواني:
1. لهذا العميل؟ chatEnabled = true هل AppSync نسأل
2. AppSync بيرجع true أو false
3. UI نحدّث الـ

استخدام الـ Polling في client_screen.dart

```
class _ClientScreenState extends State<ClientScreen> {
  Timer? _chatPollingTimer;
  bool _chatEnabled = false;

  @override
  void initState() {
    super.initState();
    _startPolling();
  }

  void _startPolling() {
    // نتحقق فوراً أول مرة
    _checkChatStatus();
  }
}
```

```

    // ثم كل 5 ثواني
    _chatPollingTimer = Timer.periodic(
      const Duration(seconds: 5),
      (_) => _checkChatStatus(),
    );
  }

Future<void> _checkChatStatus() async {
  if (!mounted) return;

  try {
    final enabled = await AWSStorageService.isChatEnabled(widget.email);

    if (mounted && enabled != _chatEnabled) {
      setState(() => _chatEnabled = enabled);

      // لو الشات اتفتح للتو
      if (enabled && !_chatEnabled) {
        _showChatOpenedDialog();
      }
    }
  } catch (e) {
    safePrint('Polling error: $e');
  }
}

@override
void dispose() {
  _chatPollingTimer?.cancel(); // مهم جداً!
  super.dispose();
}
}

```

مقارنة: Subscriptions vs Polling

الجانب	Subscriptions	Polling
السرعة	فوري (milliseconds)	interval (5 ثواني) بعد ما يجي الـ
استهلاك الباند	أقل	أكثر
استهلاك البطارية	أقل	أكثر
الموثوقية	قد تنقطع	دائماً شغال
التعقيد	أعلى	أبسط
الاستخدام في GENZ	Primary	Fallback

GENZ الاستراتيجية المثلى في

```

void _initializeRealtimeFeatures() async {
  // أولاً Subscriptions جَرِّب الـ 1.
  try {
    _subscription = AWSStorageService
      .subscribeToChatMessages(clientEmail: email)
      .listen((msg) {
        _handleNewMessage(msg);
      });

    safePrint('Subscriptions شغالة ✅');
  } catch (e) {
    safePrint('Polling فشلت ❌، بنشغل Subscriptions');

    // fallback Polling لو فشلت، اعمل 2.
    _startPolling();
  }
}

```

Polling إيقاف الـ

```

@override
void dispose() {
  // Timer وقف الـ
  _chatPollingTimer?.cancel();

  // Subscriptions وقف الـ
  _subscription?.cancel();

  super.dispose();
}

```

؟.cancel() بدل .cancel() لماذا

```

_chatPollingTimer?.cancel()
// "بس null الـ ؟. تعني" لو مش
// مش هيحصل خطأ → (== null) لم يُنشأ _chatPollingTimer لو
// آمن أكثر

```

للإشعارات Polling

```

// للإشعارات كل 10 ثواني Polling
Timer.periodic(const Duration(seconds: 10), (_) async {
  if (!mounted) return;

```

```

final newNotifs = await AWSStorageService.getNotifications(email);

if (mounted) {
  setState(() {
    // مقارنة القائمة الجديدة بالقديم
    final oldIds = _notifications.map((n) => n.id).toSet();
    final addedNotifs = newNotifs.where((n) => !oldIds.contains(n.id)).toList();

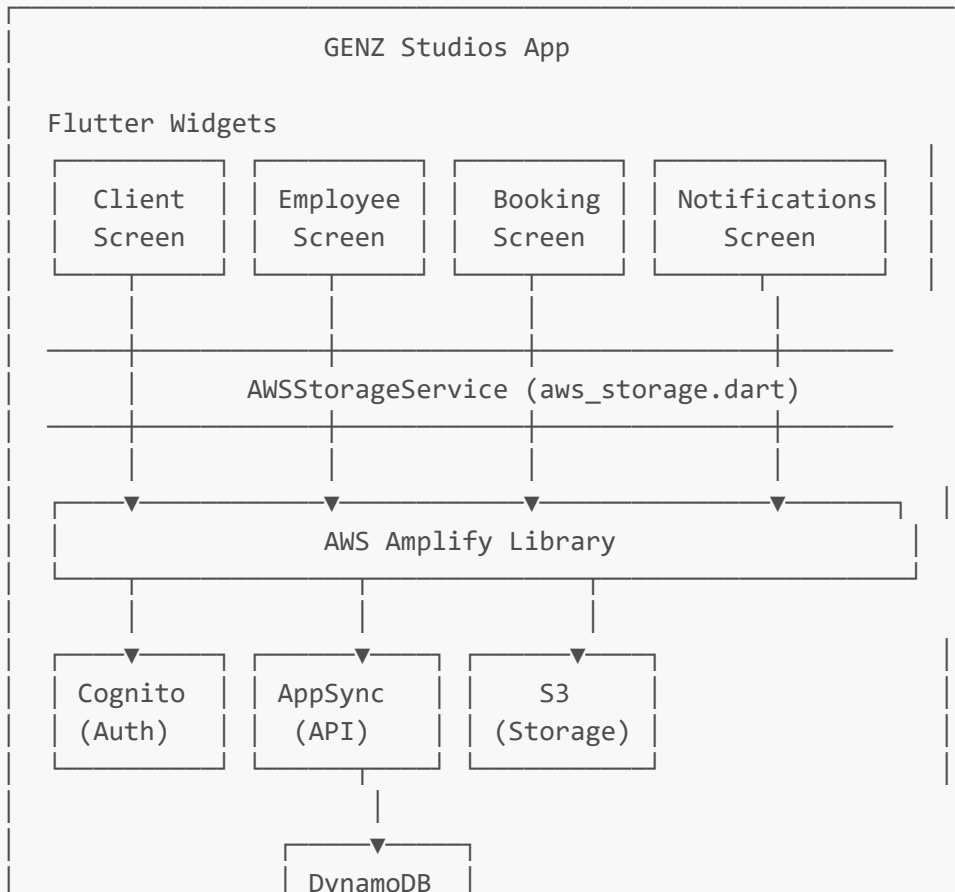
    if (addedNotifs.isNotEmpty) {
      _notifications.insertAll(0, addedNotifs);
      // عرض notification badge
      _showNotificationBadge(addedNotifs.length);
    }
  });
}
});

```

لما ال fallback السؤال الدوري عن البيانات الجديدة. بنستخدمه كـ Polling = ملخص القسم الرابع عشر
 memory leaks. عشان تجنب ال dispose() في Timer مش شغالة. مهم جداً: وقف ال Subscriptions ال
 Timer. في setState حماية ضرورية قبل أي `return (!mounted)`

الملخص العام للمشروع

خريطة كل الخدمات



(Database)

DynamoDB جدول كل الجداول في

الجدول	الهدف	الحقول الأساسية
Studio	بيانات الاستوديوهات	id, name, type, pricePerHour
BookingRequest	طلبات الحجز	id, clientEmail, studio, status
ChatMessage	رسائل الشات	id, clientEmail, senderEmail, text
AppNotification	الإشعارات	id, clientEmail, title, type, read
UserProfile	ملفات المستخدمين	id, email, chatEnabled

جدول تدفق البيانات

الحدث	من	لمن	نوع العملية
حجز جديد	العميل	DynamoDB	Mutation (create BookingRequest)
إشعار للموظفين	العميل	__employees__	Mutation (create AppNotification)
قبول/رفض الحجز	الموظف	DynamoDB	Mutation (update BookingRequest)
إشعار للعميل	الموظف	إيميل العميل	Mutation (create AppNotification)
فتح الشات	الموظف	UserProfile	Mutation (update chatEnabled)
رسالة شات	أي طرف	DynamoDB	Mutation (create ChatMessage)
جلب الاستوديوهات	التطبيق	DynamoDB	Query
جلب الإشعارات	التطبيق	DynamoDB	Query (byClientEmail)

نصائح للمبتدئين

الأخطاء الشائعة وكيفية تجنبها

1. Schema بعد تعديل الـ `amplify push` نسيت التغييرات موجودة في الكود بس مش في السحابة
→ دائماً بعد أي تعديل `amplify push` :الحل
2. push بعد الـ `amplify codegen models` نسيت مش متحدثه Dart models الـ
→ push بعد كل `amplify codegen models` :الحل
3. `update` بدون `_version` استخدمت (Conflict error) بيرفض التحديث AppSync
→ `update` الحالي قبل الـ `_version` :الحل دائماً اجلب الـ

4. Timers والـ Subscriptions للـ dispose نسيت
 - widget على setState وأخطاء Memory leak
 - dispose() في cancel الحل: دائماً
5. configureAmplify قبل runApp تشغيل
 - خطأ في بدء التطبيق
 - runApp أول ثم configureAmplify :الحل

Debug أوامر مفيدة للـ

```
# التطبيق logs مشاهدة
flutter run --verbose

# Dart models تحديث الـ
amplify codegen models

# Backend مشاهدة حالة الـ
amplify status

# Amplify CLI تحديث
npm install -g @aws-amplify/cli

# AppSync Console فتح
amplify console api

# Cognito Console فتح
amplify console auth
```

كله في السحابة Backend الـ AWS Amplify متكامل مبني على Flutter هو تطبيق GENZ Studios **الكلمة الأخيرة**: GraphQL مبني على real-time للصور. الـ S3، لقاعدة البيانات DynamoDB، API للـ AppSync، للمصادقة Cognito Push نظام الإشعارات والشات مبني بالكامل داخل التطبيق بدون fallback كـ Polling مع Subscriptions `aws_storage.dart` خارجية. كل العمليات مركزة في ملف Notifications

جميع الحقوق محفوظة — GENZ Studios تم إعداد هذا التوثيق لمشروع